

Hands On WEEK-9 DAY-42 – Keerthivasan R

Modern applications (Web, Mobile, Microservices) heavily rely on **RESTful APIs** for communication between frontend and backend systems.

In a .NET Web API project (for example: Employee Management System / Insurance Claim System), APIs are responsible for:

- User authentication
- CRUD operations
- Data validation
- Business logic processing

If APIs are not properly tested, it may lead to:

- ✗ Incorrect data processing
 - ✗ Security vulnerabilities
 - ✗ Application crashes
 - ✗ Integration failures
 - ✗ Poor user experience
-



Problem Statement

The organization has developed multiple REST APIs using ASP.NET Core Web API. However:

- There is no proper validation of request and response.
- Developers are manually checking APIs using browser or incomplete methods.
- Error responses are not consistently verified.
- Authentication tokens are not properly tested.
- Edge cases (invalid inputs, missing parameters) are not validated.

The organization needs a reliable tool to:

- Test API endpoints
 - Validate HTTP methods (GET, POST, PUT, DELETE)
 - Verify status codes
 - Test authentication (JWT tokens)
 - Automate API test cases
 - Ensure API performance and reliability
-



Proposed Solution

Use **Postman** for API Testing and Validation.

Postman will be used to:

1. Send HTTP requests to API endpoints.

2. Validate response status codes (200, 201, 400, 401, 404, 500).
 3. Test request body and parameters.
 4. Validate JSON response structure.
 5. Test JWT-based authentication.
 6. Create automated test scripts.
 7. Organize APIs into collections.
 8. Run automated test suites using Collection Runner.
-

Functional Requirements

- Test all CRUD endpoints.
 - Validate request and response payload.
 - Test authentication and authorization.
 - Check error handling scenarios.
 - Automate test cases using Postman scripts.
 - Generate test reports.
-

Non-Functional Requirements

- Ensure API reliability
 - Validate security handling
 - Maintain performance standards
 - Ensure proper error messaging
-

Tools & Technologies

- ASP.NET Core Web API
 - SQL Server
 - JWT Authentication
 - Postman
 - JSON
-

Expected Outcome

After implementing API testing using Postman:

- APIs will be fully validated.
- Bugs will be identified early.
- Security testing will improve.
- Integration issues will reduce.
- API reliability and quality will increase.

CODE:

 InsuranceClaimAPI/ Models/ PolicyHolder.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace InsuranceClaimAPI.Models
{
    public class PolicyHolder
    {
        [Key]
        public int Id { get; set; }

        [Required]
        [MaxLength(50)]
        public string PolicyNumber { get; set; } = string.Empty;

        [Required]
        [MaxLength(100)]
        public string FirstName { get; set; } = string.Empty;

        [Required]
        [MaxLength(100)]
        public string LastName { get; set; } = string.Empty;

        [Required]
        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        [Required]
        [Phone]
        public string PhoneNumber { get; set; } = string.Empty;

        [Required]
        public string Address { get; set; } = string.Empty;

        [Required]
        public string PolicyType { get; set; } = string.Empty; // Health, Auto, Home, Life

        [Column(TypeName = "decimal(18,2)")]
        public decimal CoverageAmount { get; set; }

        public DateTime PolicyStartDate { get; set; }

        public DateTime PolicyEndDate { get; set; }

        public bool IsActive { get; set; } = true;

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        // Navigation property
        public ICollection<InsuranceClaim> Claims { get; set; } = new List<InsuranceClaim>();
    }
}
```

 InsuranceClaimAPI/ Models/ InsuranceClaim.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace InsuranceClaimAPI.Models
{
    public class InsuranceClaim
    {
        [Key]
```

```

public int Id { get; set; }

[Required]
public string ClaimNumber { get; set; } = string.Empty;

[Required]
public int PolicyHolderId { get; set; }

[ForeignKey("PolicyHolderId")]
public virtual PolicyHolder? PolicyHolder { get; set; }

[Required]
public string ClaimType { get; set; } = string.Empty; // Accident, Theft, Natural
Disaster, Health

[Required]
[Column(TypeName = "decimal(18,2)")]
public decimal ClaimAmount { get; set; }

[Required]
public DateTime IncidentDate { get; set; }

[Required]
public string IncidentDescription { get; set; } = string.Empty;

public string? DocumentUrls { get; set; } // Comma-separated URLs

[Required]
public string ClaimStatus { get; set; } = "Submitted"; // Submitted, UnderReview,
Approved, Rejected, Paid

public string? Remarks { get; set; }

public DateTime? ApprovalDate { get; set; }

[Column(TypeName = "decimal(18,2)")]
public decimal? ApprovedAmount { get; set; }

public DateTime SubmittedDate { get; set; } = DateTime.UtcNow;

public DateTime? LastUpdatedDate { get; set; }

public string? ProcessedBy { get; set; }
}
}

```

InsuranceClaimAPI/ Models/ User.cs

```

using System.ComponentModel.DataAnnotations;

namespace InsuranceClaimAPI.Models
{
    public class User
    {
        [Key]
        public int Id { get; set; }

        [Required]
        [MaxLength(50)]
        public string Username { get; set; } = string.Empty;

        [Required]
        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        [Required]
        public string PasswordHash { get; set; } = string.Empty;
    }
}

```

```

        [Required]
        public string Role { get; set; } = "PolicyHolder"; // Admin, ClaimsProcessor,
PolicyHolder

        public string? RefreshToken { get; set; }
        public DateTime? RefreshTokenExpiryTime { get; set; }

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        public bool IsActive { get; set; } = true;
    }
}

```

InsuranceClaimAPI/ Models/ ClaimDocument.cs

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace InsuranceClaimAPI.Models
{
    public class ClaimDocument
    {
        [Key]
        public int Id { get; set; }

        [Required]
        public int ClaimId { get; set; }

        [ForeignKey("ClaimId")]
        public virtual InsuranceClaim? Claim { get; set; }

        [Required]
        public string DocumentName { get; set; } = string.Empty;

        [Required]
        public string DocumentUrl { get; set; } = string.Empty;

        public string? DocumentType { get; set; } // Police Report, Medical Report, Photo,
Invoice

        public DateTime UploadedAt { get; set; } = DateTime.UtcNow;

        public string? UploadedBy { get; set; }
    }
}

```

InsuranceClaimAPI/ DTOs/ LoginDto.cs

```

using System.ComponentModel.DataAnnotations;

namespace InsuranceClaimAPI.DTOs
{
    public class LoginDto
    {
        [Required]
        public string Username { get; set; } = string.Empty;

        [Required]
        public string Password { get; set; } = string.Empty;
    }

    public class RegisterDto
    {
        [Required]
        public string Username { get; set; } = string.Empty;

        [Required]

```

```

        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        [Required]
        [MinLength(6)]
        public string Password { get; set; } = string.Empty;

        [Required]
        public string Role { get; set; } = "PolicyHolder";
    }
}

```

InsuranceClaimAPI/ DTOs/ PolicyHolderDto.cs

```

namespace InsuranceClaimAPI.DTOs
{
    public class PolicyHolderDto
    {
        public int Id { get; set; }
        public string PolicyNumber { get; set; } = string.Empty;
        public string FirstName { get; set; } = string.Empty;
        public string LastName { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string PhoneNumber { get; set; } = string.Empty;
        public string Address { get; set; } = string.Empty;
        public string PolicyType { get; set; } = string.Empty;
        public decimal CoverageAmount { get; set; }
        public DateTime PolicyStartDate { get; set; }
        public DateTime PolicyEndDate { get; set; }
        public bool IsActive { get; set; }
    }

    public class CreatePolicyHolderDto
    {
        public string PolicyNumber { get; set; } = string.Empty;
        public string FirstName { get; set; } = string.Empty;
        public string LastName { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string PhoneNumber { get; set; } = string.Empty;
        public string Address { get; set; } = string.Empty;
        public string PolicyType { get; set; } = string.Empty;
        public decimal CoverageAmount { get; set; }
        public DateTime PolicyStartDate { get; set; }
        public DateTime PolicyEndDate { get; set; }
    }

    public class UpdatePolicyHolderDto
    {
        public string FirstName { get; set; } = string.Empty;
        public string LastName { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string PhoneNumber { get; set; } = string.Empty;
        public string Address { get; set; } = string.Empty;
        public bool IsActive { get; set; }
    }
}

```

InsuranceClaimAPI/ DTOs/ InsuranceClaimDto.cs

```
namespace InsuranceClaimAPI.DTOs
{
    public class InsuranceClaimDto
    {
        public int Id { get; set; }
        public string ClaimNumber { get; set; } = string.Empty;
        public int PolicyHolderId { get; set; }
        public string PolicyHolderName { get; set; } = string.Empty;
        public string ClaimType { get; set; } = string.Empty;
        public decimal ClaimAmount { get; set; }
        public DateTime IncidentDate { get; set; }
        public string IncidentDescription { get; set; } = string.Empty;
        public string ClaimStatus { get; set; } = string.Empty;
        public DateTime SubmittedDate { get; set; }
        public decimal? ApprovedAmount { get; set; }
    }

    public class CreateClaimDto
    {
        public int PolicyHolderId { get; set; }
        public string ClaimType { get; set; } = string.Empty;
        public decimal ClaimAmount { get; set; }
        public DateTime IncidentDate { get; set; }
        public string IncidentDescription { get; set; } = string.Empty;
        public string? DocumentUrls { get; set; }
    }

    public class UpdateClaimStatusDto
    {
        public string ClaimStatus { get; set; } = string.Empty; // UnderReview, Approved,
Rejected, Paid
        public decimal? ApprovedAmount { get; set; }
        public string? Remarks { get; set; }
    }
}
```

InsuranceClaimAPI/ Data/ ApplicationDbContext.cs

```
using InsuranceClaimAPI.Models;
using Microsoft.EntityFrameworkCore;

namespace InsuranceClaimAPI.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<User> Users { get; set; }
        public DbSet<PolicyHolder> PolicyHolders { get; set; }
        public DbSet<InsuranceClaim> InsuranceClaims { get; set; }
        public DbSet<ClaimDocument> ClaimDocuments { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            // Configure relationships
            modelBuilder.Entity<InsuranceClaim>()
                .HasOne(c => c.PolicyHolder)
                .WithMany(p => p.Claims)
                .HasForeignKey(c => c.PolicyHolderId)
                .OnDelete(DeleteBehavior.Restrict);
        }
    }
}
```

```

// Configure unique constraints
modelBuilder.Entity<PolicyHolder>()
    .HasIndex(p => p.PolicyNumber)
    .IsUnique();

modelBuilder.Entity<InsuranceClaim>()
    .HasIndex(c => c.ClaimNumber)
    .IsUnique();

// Seed default admin user
modelBuilder.Entity<User>().HasData(
    new User
    {
        Id = 1,
        Username = "admin",
        Email = "admin@insurance.com",
        PasswordHash = "admin123",
        Role = "Admin",
        CreatedAt = DateTime.UtcNow,
        IsActive = true
    },
    new User
    {
        Id = 2,
        Username = "claimsprocessor",
        Email = "processor@insurance.com",
        PasswordHash = "processor123",
        Role = "ClaimsProcessor",
        CreatedAt = DateTime.UtcNow,
        IsActive = true
    }
);

// Seed sample policy holder
modelBuilder.Entity<PolicyHolder>().HasData(
    new PolicyHolder
    {
        Id = 1,
        PolicyNumber = "POL-2024-001",
        FirstName = "John",
        LastName = "Smith",
        Email = "john.smith@example.com",
        PhoneNumber = "555-0101",
        Address = "123 Main St, Anytown, AN 12345",
        PolicyType = "Health",
        CoverageAmount = 500000,
        PolicyStartDate = new DateTime(2024, 1, 1),
        PolicyEndDate = new DateTime(2025, 1, 1),
        IsActive = true,
        CreatedAt = DateTime.UtcNow
    }
);
}
}
}
}
}

```

InsuranceClaimAPI/ Services/ AuthService.cs

```

using InsuranceClaimAPI.Models;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Security.Cryptography;
using System.Text;

namespace InsuranceClaimAPI.Services
{
    public interface IAuthService

```



```

{
    string GenerateJwtToken(User user);
    string GenerateRefreshToken();
    bool VerifyPassword(string password, string passwordHash);
    string HashPassword(string password);
}

public class AuthService : IAuthService
{
    private readonly IConfiguration _configuration;

    public AuthService(IConfiguration configuration)
    {
        _configuration = configuration;
    }

    public string GenerateJwtToken(User user)
    {
        var jwtSettings = _configuration.GetSection("JwtSettings");
        var secretKey = jwtSettings["Secret"];

        if (string.IsNullOrEmpty(secretKey) || secretKey.Length < 32)
        {
            throw new Exception("JWT Secret key must be at least 32 characters long");
        }

        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secretKey));
        var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var claims = new[]
        {
            new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
            new Claim(ClaimTypes.Name, user.Username),
            new Claim(ClaimTypes.Email, user.Email),
            new Claim(ClaimTypes.Role, user.Role),
            new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
            new Claim(JwtRegisteredClaimNames.Iat,
DateTimeOffset.UtcNow.ToUnixTimeSeconds().ToString(), ClaimValueTypes.Integer64)
        };

        var token = new JwtSecurityToken(
            issuer: jwtSettings["Issuer"],
            audience: jwtSettings["Audience"],
            claims: claims,
            expires:
DateTime.UtcNow.AddMinutes(double.Parse(jwtSettings["ExpiryMinutes"]!)),
            signingCredentials: credentials
        );

        return new JwtSecurityTokenHandler().WriteToken(token);
    }

    public string GenerateRefreshToken()
    {
        var randomNumber = new byte[32];
        using var rng = RandomNumberGenerator.Create();
        rng.GetBytes(randomNumber);
        return Convert.ToBase64String(randomNumber);
    }

    public bool VerifyPassword(string password, string passwordHash)
    {
        // In production, use BCrypt
        return password == passwordHash;
    }

    public string HashPassword(string password)
    {
        // In production, use BCrypt
    }
}

```

```

        return password;
    }
}
}

```

InsuranceClaimAPI/ Controllers/ AuthController.cs

```

using InsuranceClaimAPI.Data;
using InsuranceClaimAPI.DTOs;
using InsuranceClaimAPI.Models;
using InsuranceClaimAPI.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Security.Claims;

namespace InsuranceClaimAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class AuthController : ControllerBase
    {
        private readonly ApplicationDbContext _context;
        private readonly IAuthService _authService;

        public AuthController(ApplicationDbContext context, IAuthService authService)
        {
            _context = context;
            _authService = authService;
        }

        [HttpPost("login")]
        public async Task<IActionResult> Login([FromBody] LoginDto loginDto)
        {
            if (!ModelState.IsValid)
                return BadRequest(ModelState);

            var user = await _context.Users
                .FirstOrDefaultAsync(u => u.Username == loginDto.Username && u.IsActive);

            if (user == null)
                return Unauthorized(new { message = "Invalid username or password" });

            if (!_authService.VerifyPassword(loginDto.Password, user.PasswordHash))
                return Unauthorized(new { message = "Invalid username or password" });

            var accessToken = _authService.GenerateJwtToken(user);
            var refreshToken = _authService.GenerateRefreshToken();

            user.RefreshToken = refreshToken;
            user.RefreshTokenExpiryTime = DateTime.UtcNow.AddDays(7);
            await _context.SaveChangesAsync();

            return Ok(new
            {
                AccessToken = accessToken,
                RefreshToken = refreshToken,
                ExpiresIn = 3600,
                Username = user.Username,
                Email = user.Email,
                Role = user.Role
            });
        }

        [HttpPost("register")]
        public async Task<IActionResult> Register([FromBody] RegisterDto registerDto)
        {
            if (!ModelState.IsValid)

```

```

        return BadRequest(ModelState);

        if (await _context.Users.AnyAsync(u => u.Username == registerDto.Username))
            return BadRequest(new { message = "Username already exists" });

        if (await _context.Users.AnyAsync(u => u.Email == registerDto.Email))
            return BadRequest(new { message = "Email already exists" });

        var user = new User
        {
            Username = registerDto.Username,
            Email = registerDto.Email,
            PasswordHash = _authService.HashPassword(registerDto.Password),
            Role = registerDto.Role,
            CreatedAt = DateTime.UtcNow,
            IsActive = true
        };

        _context.Users.Add(user);
        await _context.SaveChangesAsync();

        return Ok(new { message = "User registered successfully", username = user.Username
    });
}

[Authorize]
[HttpPost("logout")]
public async Task<IActionResult> Logout()
{
    var userId = int.Parse(User.FindFirst(ClaimTypes.NameIdentifier)?.Value ?? "0");
    var user = await _context.Users.FindAsync(userId);

    if (user != null)
    {
        user.RefreshToken = null;
        user.RefreshTokenExpiryTime = null;
        await _context.SaveChangesAsync();
    }

    return Ok(new { message = "Logged out successfully" });
}

[Authorize]
[HttpGet("me")]
public IActionResult GetCurrentUser()
{
    var userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
    var userName = User.FindFirst(ClaimTypes.Name)?.Value;
    var email = User.FindFirst(ClaimTypes.Email)?.Value;
    var role = User.FindFirst(ClaimTypes.Role)?.Value;

    return Ok(new
    {
        UserId = userId,
        Username = userName,
        Email = email,
        Role = role
    });
}
}
}

```

 InsuranceClaimAPI/ Controllers/ PolicyHoldersController.cs

```

using InsuranceClaimAPI.Data;
using InsuranceClaimAPI.DTOs;
using InsuranceClaimAPI.Models;
using Microsoft.AspNetCore.Authorization;

```

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace InsuranceClaimAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    [Authorize]
    public class PolicyHoldersController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public PolicyHoldersController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: api/policyholders
        [HttpGet]
        [Authorize(Roles = "Admin,ClaimsProcessor")]
        public async Task<ActionResult<IEnumerable<PolicyHolderDto>>> GetPolicyHolders()
        {
            var policyHolders = await _context.PolicyHolders
                .Select(p => new PolicyHolderDto
                {
                    Id = p.Id,
                    PolicyNumber = p.PolicyNumber,
                    FirstName = p.FirstName,
                    LastName = p.LastName,
                    Email = p.Email,
                    PhoneNumber = p.PhoneNumber,
                    Address = p.Address,
                    PolicyType = p.PolicyType,
                    CoverageAmount = p.CoverageAmount,
                    PolicyStartDate = p.PolicyStartDate,
                    PolicyEndDate = p.PolicyEndDate,
                    IsActive = p.IsActive
                })
                .ToListAsync();

            return Ok(policyHolders);
        }

        // GET: api/policyholders/5
        [HttpGet("{id}")]
        public async Task<ActionResult<PolicyHolderDto>> GetPolicyHolder(int id)
        {
            var policyHolder = await _context.PolicyHolders.FindAsync(id);

            if (policyHolder == null)
                return NotFound(new { message = $"Policy holder with ID {id} not found" });

            var policyHolderDto = new PolicyHolderDto
            {
                Id = policyHolder.Id,
                PolicyNumber = policyHolder.PolicyNumber,
                FirstName = policyHolder.FirstName,
                LastName = policyHolder.LastName,
                Email = policyHolder.Email,
                PhoneNumber = policyHolder.PhoneNumber,
                Address = policyHolder.Address,
                PolicyType = policyHolder.PolicyType,
                CoverageAmount = policyHolder.CoverageAmount,
                PolicyStartDate = policyHolder.PolicyStartDate,
                PolicyEndDate = policyHolder.PolicyEndDate,
                IsActive = policyHolder.IsActive
            };

            return Ok(policyHolderDto);
        }
    }
}

```

```

    }

    // POST: api/policyholders
    [HttpPost]
    [Authorize(Roles = "Admin")]
    public async Task<ActionResult<PolicyHolderDto>> CreatePolicyHolder([FromBody]
CreatePolicyHolderDto createDto)
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);

        if (await _context.PolicyHolders.AnyAsync(p => p.PolicyNumber ==
createDto.PolicyNumber))
            return BadRequest(new { message = "Policy number already exists" });

        if (await _context.PolicyHolders.AnyAsync(p => p.Email == createDto.Email))
            return BadRequest(new { message = "Email already exists" });

        var policyHolder = new PolicyHolder
        {
            PolicyNumber = createDto.PolicyNumber,
            FirstName = createDto.FirstName,
            LastName = createDto.LastName,
            Email = createDto.Email,
            PhoneNumber = createDto.PhoneNumber,
            Address = createDto.Address,
            PolicyType = createDto.PolicyType,
            CoverageAmount = createDto.CoverageAmount,
            PolicyStartDate = createDto.PolicyStartDate,
            PolicyEndDate = createDto.PolicyEndDate,
            IsActive = true,
            CreatedAt = DateTime.UtcNow
        };

        _context.PolicyHolders.Add(policyHolder);
        await _context.SaveChangesAsync();

        var policyHolderDto = new PolicyHolderDto
        {
            Id = policyHolder.Id,
            PolicyNumber = policyHolder.PolicyNumber,
            FirstName = policyHolder.FirstName,
            LastName = policyHolder.LastName,
            Email = policyHolder.Email,
            PhoneNumber = policyHolder.PhoneNumber,
            Address = policyHolder.Address,
            PolicyType = policyHolder.PolicyType,
            CoverageAmount = policyHolder.CoverageAmount,
            PolicyStartDate = policyHolder.PolicyStartDate,
            PolicyEndDate = policyHolder.PolicyEndDate,
            IsActive = policyHolder.IsActive
        };

        return CreatedAtAction(nameof(GetPolicyHolder), new { id = policyHolder.Id },
policyHolderDto);
    }

    // PUT: api/policyholders/5
    [HttpPut("{id}")]
    [Authorize(Roles = "Admin")]
    public async Task<IActionResult> UpdatePolicyHolder(int id, [FromBody]
UpdatePolicyHolderDto updateDto)
    {
        var policyHolder = await _context.PolicyHolders.FindAsync(id);

        if (policyHolder == null)
            return NotFound(new { message = $"Policy holder with ID {id} not found" });

        if (policyHolder.Email != updateDto.Email &&

```

```

        await _context.PolicyHolders.AnyAsync(p => p.Email == updateDto.Email))
        return BadRequest(new { message = "Email already exists" });

        policyHolder.FirstName = updateDto.FirstName;
        policyHolder.LastName = updateDto.LastName;
        policyHolder.Email = updateDto.Email;
        policyHolder.PhoneNumber = updateDto.PhoneNumber;
        policyHolder.Address = updateDto.Address;
        policyHolder.IsActive = updateDto.IsActive;

        await _context.SaveChangesAsync();

        return Ok(new { message = "Policy holder updated successfully" });
    }

    // DELETE: api/policyholders/5
    [HttpDelete("{id}")]
    [Authorize(Roles = "Admin")]
    public async Task<IActionResult> DeletePolicyHolder(int id)
    {
        var policyHolder = await _context.PolicyHolders
            .Include(p => p.Claims)
            .FirstOrDefaultAsync(p => p.Id == id);

        if (policyHolder == null)
            return NotFound(new { message = $"Policy holder with ID {id} not found" });

        if (policyHolder.Claims.Any())
            return BadRequest(new { message = "Cannot delete policy holder with existing claims" });

        _context.PolicyHolders.Remove(policyHolder);
        await _context.SaveChangesAsync();

        return Ok(new { message = "Policy holder deleted successfully" });
    }
}

```

InsuranceClaimAPI/ Controllers/ ClaimsController.cs

```

using InsuranceClaimAPI.Data;
using InsuranceClaimAPI.DTOs;
using InsuranceClaimAPI.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Security.Claims;

namespace InsuranceClaimAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    [Authorize]
    public class ClaimsController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public ClaimsController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: api/claims
        [HttpGet]
        [Authorize(Roles = "Admin, ClaimsProcessor")]
        public async Task<ActionResult<IEnumerable<InsuranceClaimDto>>> GetAllClaims()
        {

```

```

var claims = await _context.InsuranceClaims
    .Include(c => c.PolicyHolder)
    .Select(c => new InsuranceClaimDto
    {
        Id = c.Id,
        ClaimNumber = c.ClaimNumber,
        PolicyHolderId = c.PolicyHolderId,
        PolicyHolderName = $"{c.PolicyHolder!.FirstName}
{c.PolicyHolder.LastName}",
        ClaimType = c.ClaimType,
        ClaimAmount = c.ClaimAmount,
        IncidentDate = c.IncidentDate,
        IncidentDescription = c.IncidentDescription,
        ClaimStatus = c.ClaimStatus,
        SubmittedDate = c.SubmittedDate,
        ApprovedAmount = c.ApprovedAmount
    })
    .ToListAsync();

return Ok(claims);
}

// GET: api/claims/myclaims
[HttpGet("myclaims")]
public async Task<ActionResult<IEnumerable<InsuranceClaimDto>>> GetMyClaims()
{
    var userId = int.Parse(User.FindFirst(ClaimTypes.NameIdentifier)?.Value ?? "0");
    var user = await _context.Users.FindAsync(userId);

    // For PolicyHolder role, get claims linked to their policy holder record
    // For simplicity, we'll return all claims (you can enhance this logic)
    var claims = await _context.InsuranceClaims
        .Include(c => c.PolicyHolder)
        .Select(c => new InsuranceClaimDto
        {
            Id = c.Id,
            ClaimNumber = c.ClaimNumber,
            PolicyHolderId = c.PolicyHolderId,
            PolicyHolderName = $"{c.PolicyHolder!.FirstName}
{c.PolicyHolder.LastName}",
            ClaimType = c.ClaimType,
            ClaimAmount = c.ClaimAmount,
            IncidentDate = c.IncidentDate,
            IncidentDescription = c.IncidentDescription,
            ClaimStatus = c.ClaimStatus,
            SubmittedDate = c.SubmittedDate,
            ApprovedAmount = c.ApprovedAmount
        })
        .ToListAsync();

    return Ok(claims);
}

// GET: api/claims/5
[HttpGet("{id}")]
public async Task<ActionResult<InsuranceClaimDto>> GetClaim(int id)
{
    var claim = await _context.InsuranceClaims
        .Include(c => c.PolicyHolder)
        .FirstOrDefaultAsync(c => c.Id == id);

    if (claim == null)
        return NotFound(new { message = $"Claim with ID {id} not found" });

    var claimDto = new InsuranceClaimDto
    {
        Id = claim.Id,
        ClaimNumber = claim.ClaimNumber,
        PolicyHolderId = claim.PolicyHolderId,

```

```

        PolicyHolderName = $"{claim.PolicyHolder!.FirstName}
{claim.PolicyHolder.LastName}",
        ClaimType = claim.ClaimType,
        ClaimAmount = claim.ClaimAmount,
        IncidentDate = claim.IncidentDate,
        IncidentDescription = claim.IncidentDescription,
        ClaimStatus = claim.ClaimStatus,
        SubmittedDate = claim.SubmittedDate,
        ApprovedAmount = claim.ApprovedAmount
    };

    return Ok(claimDto);
}

// POST: api/claims
[HttpPost]
public async Task<ActionResult<InsuranceClaimDto>> CreateClaim([FromBody]
CreateClaimDto createDto)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);

    var policyHolder = await
_context.PolicyHolders.FindAsync(createDto.PolicyHolderId);
    if (policyHolder == null)
        return BadRequest(new { message = "Policy holder not found" });

    // Generate unique claim number
    var claimNumber = $"CLM-{DateTime.Now:yyyyMMdd}-{new Random().Next(1000, 9999)}";

    var claim = new InsuranceClaim
    {
        ClaimNumber = claimNumber,
        PolicyHolderId = createDto.PolicyHolderId,
        ClaimType = createDto.ClaimType,
        ClaimAmount = createDto.ClaimAmount,
        IncidentDate = createDto.IncidentDate,
        IncidentDescription = createDto.IncidentDescription,
        DocumentUrls = createDto.DocumentUrls,
        ClaimStatus = "Submitted",
        SubmittedDate = DateTime.UtcNow,
        LastUpdatedDate = DateTime.UtcNow
    };

    _context.InsuranceClaims.Add(claim);
    await _context.SaveChangesAsync();

    var claimDto = new InsuranceClaimDto
    {
        Id = claim.Id,
        ClaimNumber = claim.ClaimNumber,
        PolicyHolderId = claim.PolicyHolderId,
        PolicyHolderName = $"{policyHolder.FirstName} {policyHolder.LastName}",
        ClaimType = claim.ClaimType,
        ClaimAmount = claim.ClaimAmount,
        IncidentDate = claim.IncidentDate,
        IncidentDescription = claim.IncidentDescription,
        ClaimStatus = claim.ClaimStatus,
        SubmittedDate = claim.SubmittedDate,
        ApprovedAmount = claim.ApprovedAmount
    };

    return CreatedAtAction(nameof(GetClaim), new { id = claim.Id }, claimDto);
}

// PUT: api/claims/5/status
[HttpPut("{id}/status")]
[Authorize(Roles = "Admin,ClaimsProcessor")]

```



```

    public async Task<IActionResult> UpdateClaimStatus(int id, [FromBody]
UpdateClaimStatusDto updateDto)
    {
        var claim = await _context.InsuranceClaims.FindAsync(id);

        if (claim == null)
            return NotFound(new { message = $"Claim with ID {id} not found" });

        var validStatuses = new[] { "UnderReview", "Approved", "Rejected", "Paid" };
        if (!validStatuses.Contains(updateDto.ClaimStatus))
            return BadRequest(new { message = "Invalid claim status" });

        claim.ClaimStatus = updateDto.ClaimStatus;
        claim.ApprovedAmount = updateDto.ApprovedAmount;
        claim.Remarks = updateDto.Remarks;
        claim.LastUpdatedDate = DateTime.UtcNow;
        claim.ProcessedBy = User.FindFirst(ClaimTypes.Name)?.Value;

        if (updateDto.ClaimStatus == "Approved" || updateDto.ClaimStatus == "Rejected")
        {
            claim.ApprovalDate = DateTime.UtcNow;
        }

        await _context.SaveChangesAsync();

        return Ok(new { message = $"Claim status updated to {updateDto.ClaimStatus}" });
    }

    // DELETE: api/claims/5
    [HttpDelete("{id}")]
    [Authorize(Roles = "Admin")]
    public async Task<IActionResult> DeleteClaim(int id)
    {
        var claim = await _context.InsuranceClaims.FindAsync(id);

        if (claim == null)
            return NotFound(new { message = $"Claim with ID {id} not found" });

        if (claim.ClaimStatus != "Submitted")
            return BadRequest(new { message = "Cannot delete claim that is already under
review or processed" });

        _context.InsuranceClaims.Remove(claim);
        await _context.SaveChangesAsync();

        return Ok(new { message = "Claim deleted successfully" });
    }

    // GET: api/claims/statistics
    [HttpGet("statistics")]
    [Authorize(Roles = "Admin, ClaimsProcessor")]
    public async Task<IActionResult> GetClaimStatistics()
    {
        var totalClaims = await _context.InsuranceClaims.CountAsync();
        var approvedClaims = await _context.InsuranceClaims.CountAsync(c => c.ClaimStatus
== "Approved");
        var rejectedClaims = await _context.InsuranceClaims.CountAsync(c => c.ClaimStatus
== "Rejected");
        var pendingClaims = await _context.InsuranceClaims.CountAsync(c => c.ClaimStatus
== "Submitted" || c.ClaimStatus == "UnderReview");
        var totalAmount = await _context.InsuranceClaims.SumAsync(c => c.ClaimAmount);
        var approvedAmount = await _context.InsuranceClaims.SumAsync(c => c.ApprovedAmount
?? 0);

        return Ok(new
        {
            TotalClaims = totalClaims,
            ApprovedClaims = approvedClaims,
            RejectedClaims = rejectedClaims,

```

```

        PendingClaims = pendingClaims,
        TotalClaimAmount = totalAmount,
        TotalApprovedAmount = approvedAmount
    });
}
}
}

```

InsuranceClaimAPI/ appSettings.json

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\mssqllocaldb;Database=InsuranceClaimDB;Trusted_Connection=True;MultipleActiveResultSets=true"
  },
  "JwtSettings": {
    "Secret": "InsuranceClaimSystemSuperSecretKey2024!@#$$%^&*",
    "Issuer": "InsuranceClaimAPI",
    "Audience": "InsuranceClaimClient",
    "ExpiryMinutes": 60
  }
}

```

InsuranceClaimAPI/ Program.cs

```

using InsuranceClaimAPI.Data;
using InsuranceClaimAPI.Services;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

// Add services
builder.Services.AddControllers();

// Configure Swagger
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo
    {
        Title = "Insurance Claim Management API",
        Version = "v1",
        Description = "API for managing insurance claims, policy holders, and claim
processing"
    });

    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Name = "Authorization",
        Type = SecuritySchemeType.ApiKey,
        Scheme = "Bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
        Description = "Enter 'Bearer' followed by your token. Example: Bearer
eyJhbGciOiJIUzI1NiIs..."
    });
}

```

```

});

c.AddSecurityRequirement(new OpenApiSecurityRequirement
{
    {
        new OpenApiSecurityScheme
        {
            Reference = new OpenApiReference
            {
                Type = ReferenceType.SecurityScheme,
                Id = "Bearer"
            },
            Array.Empty<string>()
        },
    }
});

});

// Configure Database
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// Configure JWT Authentication
var jwtSettings = builder.Configuration.GetSection("JwtSettings");
var secretKey = jwtSettings["Secret"];

if (string.IsNullOrEmpty(secretKey) || secretKey.Length < 32)
{
    throw new Exception("JWT Secret key must be at least 32 characters long");
}

var key = Encoding.UTF8.GetBytes(secretKey);

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.RequireHttpsMetadata = false;
    options.SaveToken = true;
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = jwtSettings["Issuer"],
        ValidAudience = jwtSettings["Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(key),
        ClockSkew = TimeSpan.Zero
    };
});

builder.Services.AddAuthorization();

// Register Services
builder.Services.AddScoped<IAuthService, AuthService>();

var app = builder.Build();

// Create database on startup
using (var scope = app.Services.CreateScope())
{
    var dbContext = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
    dbContext.Database.EnsureCreated();
    Console.WriteLine("☑ Insurance Claim Database ready");
}

```

```
// Configure pipeline
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

OUTPUT:

The screenshot displays the Swagger UI interface for a REST API. The top bar indicates a **POST** request to the endpoint `/api/Auth/login`. The **Parameters** tab shows no parameters. The **Request body** is set to `application/json` and contains the following JSON:

```
{
  "username": "admin",
  "password": "admin123"
}
```

The **Execute** button has been clicked, and the **Responses** tab is active. It shows a **200** status code with the following response body:

```
{
  "accessToken": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsInR5cGU6IiwiZW50cnVkaXkiLCJ1b29udG91Ij06ImN1b29udG91IiwiaWF0Ij06IjE5OTk5MjE0LjE1In0",
  "refreshToken": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsInR5cGU6IiwiZW50cnVkaXkiLCJ1b29udG91Ij06ImN1b29udG91IiwiaWF0Ij06IjE5OTk5MjE0LjE1In0",
  "expiresIn": 3600,
  "username": "admin",
  "email": "admin@insurance.com",
  "role": "Admin"
}
```

The **Response headers** are:

```
content-type: application/json; charset=utf-8
date: Tue, 15 Apr 2025 17:52:05 GMT
server: Kestrel
```

At the bottom, a table summarizes the response:

Code	Description	Links
200	OK	No links

POST

/api/Auth/register

Parameters

No parameters

Request body

application/json

```
{  
  "username": "Keerthivasan",  
  "email": "keerthi@sample.com",  
  "password": "keerthi123",  
  "role": "admin"  
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \\  
  https://localhost:7189/api/auth/register' \\  
  -H 'accept: */*' \\  
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpVCXB5LCJ0eXAiOiJKV1QiLCJpdGVzaWkiOiJkaWYyPSJ9eyJ0eXAiOiJKV1QiLCJpdGVzaWkiOiJkaWYyPSJ9eyJ0eXAiOiJKV1QiLCJpdGVzaWkiOiJkaWYyPSJ9' \\  
  -H 'Content-Type: application/json' \\  
  -d '{  
    "username": "Keerthivasan",  
    "email": "keerthi@sample.com",  
    "password": "keerthi123",  
    "role": "admin"  
  }'  
&
```

Request URL

https://localhost:7189/api/auth/register

Server response

Code	Details
200	Response body <pre>{ "message": "User registered successfully", "username": "Keerthivasan" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Tue, 14 Apr 2025 17:33:29 GMT server: Kestrel</pre>

Responses

Code	Description	Links
200	OK	No links

[illegible]

^



Cancel

No parameters

Clear

Cart

[illegible]

Required URL:

<https://localhost:7189/api/Classes>

Server response

Code	Details
------	---------

200

Response body

```

    "id": 1,
    "claimNumber": "CLR-20200414-0114",
    "policyholderId": 1,
    "policyholderName": "Rutha Kumar",
    "claimType": "Accident",
    "claimAmount": 5000,
    "incidentDate": "2024-03-10T00:00:00",
    "incidentDescription": "Vehicle collision at intersection. Other driver ran red light. Front bumper and headlight damage. Police report filed.",
    "claimStatus": "Submitted",
    "submittedDate": "2024-04-14T12:31.27.000Z",
    "approvedAmount": null
  },
  {
    "id": 2,
    "claimNumber": "CLR-20200414-0798",
    "policyholderId": 2,
    "policyholderName": "John Smith",
    "claimType": "Health",
    "claimAmount": 100000,
    "incidentDate": "2024-03-20T00:00:00",
    "incidentDescription": "Major surgery claim",
    "claimStatus": "Submitted",
    "submittedDate": "2024-04-14T12:31.39.670Z",
    "approvedAmount": null
  }
]

```

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 14 Apr 2018 17:54:12 GMT
server: Kestrel
```

Reporters

Code	Description
------	-------------

Links

200

200 OK

No links

Media type

text/plain

Continuity Across Periods:

Example Value	Schema
---------------	--------

```
{
  "id": 0,
  "claimNumber": "string",
  "policyHolderId": 0,
  "policyHolderName": "string",
  "claimType": "string",
  "claimAmount": 0,
  "incidentDate": "2020-04-14T17:47:23.995Z",
  "incidentDescription": "string",
  "claimStatus": "string",
  "submittedDate": "2020-04-14T17:47:23.995Z",
  "approvedAmount": 0
}
```


set

No parameters

Request body

```
application/json
```

```
"policyHolderId": 2,
"claimType": "Accident",
"claimAmount": 8000,
"incidentDate": "2024-01-18T00:00:00",
"incidentDescription": "Vehicle collision at intersection. Other driver ran red light. Front bumper and headlight damage. Police report filed.",
"documentUris": "https://storage.insurance.com/docs/Police_Report_3421.pdf,https://storage.insurance.com/photos/damage-front.jpg"
```

Clear

Responses

Cart

[illegible]

Request URL:

https://localhost:7169/api/Claims

Server response

Code	Details
------	---------

201

201 Response body

```
{
  "id": 1,
  "clientId": "CLR-20150614-2705",
  "policyHolderId": 2,
  "policyHolderName": "Rutha Kanan",
  "claimType": "Accident",
  "claimEvent": "Car",
  "incidentDate": "2014-05-10T09:00:00Z",
  "incidentDescription": "Vehicle collision at intersection. Other driver ran red light. Front bumper and headlights damage. Police report filed.",
  "claimStatus": "Submitted",
  "submittedDate": "2015-06-10T17:35:48.5661796Z",
  "approvedAmount": null
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 14 Apr 2015 17:25:48 GMT
location: https://localhost:7100/api/Claims/5
server: Kestrel
```

Responses

Code	Description
------	-------------

Links

2005

DK

No links

uint8 type

[Next/Previous](#)

Controlle des coûts, par exemple

Example	Value	Scheme
1	10	10
2	20	20
3	30	30
4	40	40
5	50	50
6	60	60
7	70	70
8	80	80
9	90	90
10	100	100

```
{
  "id": 0,
  "className": "string",
  "policyHolderId": 0,
  "policyHolderName": "string",
  "claimType": "string",
  "claimAmount": 0,
  "incidentDate": "2026-06-14T17:47:23.996Z",
  "incidentDescription": "string",
  "claimStatus": "string",
  "submittedDate": "2026-06-14T17:47:23.996Z",
  "approvedAmount": 0
}
```

GET /api/Claims/myclaims

Parameters

Cancel

Pharmacokinetics

Constructive

Clear

Responses

Conf

[illegible]

Revised 10/01

<https://localhost:7180/api/Claims/myclaims>

Code	Details
------	---------

200

```

{
  "id": 1,
  "claimNumber": "CLR-20200414-0001",
  "policyHolderId": 1,
  "policyHolderName": "John Smith",
  "claimType": "Accident",
  "claimAmount": 10000,
  "incidentDate": "2020-03-15T00:00:00",
  "incidentDescription": "Car accident on highway, rear-ended by another vehicle",
  "claimStatus": "Approved",
  "submittedDate": "2020-04-10T11:00:00",
  "approvedAmount": 10000
}

```


Gott

Request URL

<https://localhost:7100/api/Claims/1/status>

Code	Details
------	---------

200

Response Body

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 14 Apr 2020 17:43:57 GMT
server: Kestrel
```

Code	Description
------	-------------

200

OK

Links

No links

GET /api/Claims/statistics

Parameters

Cancel

No parameters

Exercises

Clear

Responses

Cyl

Request URL:

<https://insallment.7100/apl/Claims/activation>

Server response

Code Details

200

Response body

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 14 Apr 2015 17:02:56 GMT
server: Kestrel
```

Responses

Code	Description
------	-------------

200

OK

Links

No links

PolicyHolders

GET /api/PolicyHolders


```
{
  "id": 1,
  "policyNumber": "POL-2024-001",
  "firstName": "John",
  "lastName": "Smith",
  "email": "john.smith@example.com",
  "phoneNumber": "335-0123",
  "address": "123 Main St, Anytown, NY 12345",
  "policyType": "Health",
  "coverageAmount": 100000,
  "policyStartDate": "2024-01-01T00:00:00",
  "policyEndDate": "2025-01-01T00:00:00",
  "isActive": true
}
```

[illegible]Clear